PLAN DE DESPLIEGUE DE SOFTWARE

Modelo de implementación y puesta en producción | Versión 1.0

(Plantilla base: `Plan de Despliegue_SW.pdf`, aplicada al proyecto RespiCare Tacna)

Campo	Valor
Proyecto / sistema	RespiCare Tacna — Sistema Web y Móvil para la Detección de Enfermedades Respiratorias en Tacna
Organización	Universidad Privada de Tacna — Curso Construcción de Software II
Fecha	23/10/2026
Responsable del plan	Cesar Fabian Chavez Linares (Líder de despliegue / Desarrollo)
Ambiente de destino	Producción (VM propia + Cloudflare Tunnel)

1. Propósito y descripción

Establecer un procedimiento controlado, reproducible y verificable para liberar e implementar una versión de RespiCare (backend, ai-services, web, mobile) en el ambiente de producción, minimizando interrupciones, errores de configuración, pérdida de datos y riesgos de seguridad. El plan define condiciones previas, secuencia de ejecución, responsabilidades, pruebas, comunicación y reversión.

2. Objetivos

- Asegurar que la versión autorizada (commit en `fabian/main`) se instale con sus dependencias y variables de entorno correctas (`.env` de producción).
- Mantener disponibilidad, integridad, confidencialidad y trazabilidad durante el cambio (datos clínicos de pacientes).
- Validar procesos críticos (autenticación, análisis de síntomas con IA, reportes de casos) antes de habilitar el sistema a usuarios finales.
- Disponer de un procedimiento de retorno a la versión estable ante fallas relevantes.

3. Alcance y supuestos

Incluye: preparación de entornos (VM de producción), respaldo de MongoDB/Redis, distribución de artefactos (imágenes Docker construidas localmente en la VM), despliegue del stack backend/ai-services/datos vía `scripts/deploy-vm.sh` (`docker-compose.vm.yml`), despliegue separado del frontend web en Vercel (autodeploy por push), gestión del túnel público vía `scripts/tunnel-manager.sh`, pruebas técnicas y funcionales, monitoreo, soporte inicial y cierre.

No incluye: nuevas funcionalidades no aprobadas en el sprint vigente.

Supuestos:

- Existe versión candidata identificada (commit en `fabian/main` ya mergeado y con CI en verde).
- El build de imágenes de `backend` y `ai-services` se hace directamente en la VM (`docker compose -f docker-compose.vm.yml build`), no se descargan desde un registro externo.
- Acceso SSH aprobado a la VM de producción (`38.250.158.233`) y al servicio `tunnel-manager.sh/cloudflared` corriendo en ella.
- El responsable (único desarrollador activo del proyecto) está disponible durante la ventana de cambio.

Arquitectura real de despliegue (confirmada en `scripts/deploy-vm.sh`, `scripts/tunnel-manager.sh` y `nginx/nginx.vm.conf`):

- **Backend + ai-services + datos**: un único stack Docker Compose (`docker-compose.vm.yml`: `mongodb`, `redis`, `backend`, `ai-services`, `nginx`, `mongo-express`, `redis-commander`) levantado con `scripts/deploy-vm.sh`.
- **Web (frontend)**: **no** vive en la VM ni en `docker-compose.vm.yml` — está desplegado en **Vercel**. `nginx.vm.conf` solo hace reverse proxy de `/api/`, `/ai/`, `/socket.io/`, `/mongo-express`, `/redis-commander/`; no sirve ningún estático de `web/`.
- **Túnel público**: no es el contenedor `cloudflared` de `docker-compose.tunnel.yml` (ese archivo es un overlay para `docker-compose.dev.yml`, solo para pruebas locales y nunca se combina con `docker-compose.vm.yml`). En producción, `cloudflared` corre como **proceso a nivel de host** vía `scripts/tunnel-manager.sh`, apuntando a `http://localhost:80` (el puerto que expone el `nginx` del compose).
- **Rotación de URL**: el túnel es un *quick tunnel* de Cloudflare (`*.trycloudflare.com`), **efímero** — la URL cambia en cada reinicio del servicio/VM. Cuando rota, `tunnel-manager.sh` reescribe automáticamente `.env`, `.env.vm`, configs de mobile, `nginx.vm.conf` y `web/src/utils/apiBase.js`, y **hace commit + push automático a la rama `fabian`**, lo que dispara el autodeploy de Vercel. Es una automatización sin intervención humana que el responsable debe tener presente al revisar el historial de commits.

⚠️ **Nota de alcance crítica para este proyecto**: existía un workflow (`deploy-production.yml`) que desplegaba a un clúster **Kubernetes** (`KUBERNETES_NAMESPACE: respicare-production`). Ese flujo no reflejaba la infraestructura real en uso, que es la descrita arriba (Compose en VM + túnel a nivel de host + Vercel para el web); nunca se ejecutó (0 corridas, 0 tags `v*.*.*`, sin el secret `KUBECONFIG_PRODUCTION`). Se archivó en `docs/archive/workflows/deploy-production.yml` para que deje de listarse como workflow activo y no contradiga este plan. Este plan documenta el flujo **real**.

4. Estrategia de despliegue

Despliegue gradual por ambiente: **Desarrollo** (`docker-compose.dev.yml`) → **Pruebas/CI** (`.github/workflows/testing.yml`, `backend-tests.yml`, `web-tests.yml`, `ai-services-tests.yml`) → **Producción** (`docker-compose.vm.yml` vía `scripts/deploy-vm.sh` para backend/ai-services/datos, + Vercel autodeploy para el web).

No existe hoy un ambiente de preproducción separado con datos anonimizados — es un gap identificado en `CHECKLIST_DESPLIEGUE_TESTABILIDAD_QA.md`, sección 1.

Elemento	Definición para RespiCare
Tipo de despliegue	**Directo / recreate** (no hay blue-green ni rolling real implementado; `scripts/deploy-vm.sh` ejecuta `docker compose up -d --build`, que recrea los contenedores). Nombrarlo así honestamente, no como blue-green/canary. El web se despliega aparte vía autodeploy de Vercel al hacer push.
Versión / paquete	Commit/tag en la rama `fabian/main`; las imágenes de `backend`/`ai-services` se construyen en la propia VM (no hay registro intermedio tipo `ghcr.io` en este flujo)
Servicios afectados	`backend` (3001), `ai-services` (8000), `mongodb`, `redis`, `nginx` reverse proxy (puerto 80) — todos en la VM vía Compose. `web` (Vercel, independiente). `cloudflared` (proceso de host, no contenedor)
Ventana programada	A definir por el responsable antes de cada despliegue (recomendado: fuera de horario de uso activo, dado que es sistema académico no 24/7)
Interrupción esperada	Hasta ~2-3 min (tiempo de rebuild + healthcheck `start_period: 40s` backend / `90s` ai-services). El web en Vercel no sufre downtime propio del despliegue de la VM.
Criterio para avanzar	Los healthchecks de Compose deben reportar `healthy`: `mongodb`, `redis`, backend `GET /health`, ai-services `GET /api/v1/health`, nginx `GET /nginx-health` (son los mismos que valida `wait_healthy()` en `scripts/deploy-vm.sh`)

5. Roles y responsabilidades

El equipo real de este proyecto es de desarrollo individual (Cesar Fabian Chavez Linares cubre todos los roles técnicos); se mantiene la tabla de roles del estándar para que quede explícito qué responsabilidad asume cada función aunque recaiga en la misma persona.

Rol	Responsabilidad principal	Quién lo cubre en RespiCare
Líder de despliegue	Coordina, registra hitos, verifica aprobaciones, decide escalamiento	Cesar Fabian Chavez Linares
Equipo de desarrollo /	Prepara artefactos, scripts y	Cesar Fabian Chavez

DevOps	automatizaciones; ejecuta la liberación	Linares
Infraestructura / operaciones	Valida capacidad, accesos, red, certificados, monitoreo y respaldos	Cesar Fabian Chavez Linares
Calidad (QA) / usuario clave	Ejecuta pruebas de humo y valida procesos críticos	Cesar Fabian Chavez Linares (apoyado en suites automatizadas: `CATALOGO_PRUEBAS.md`)
Seguridad de la información	Revisa controles, vulnerabilidades relevantes	CI automatizado: `dast-scan.yml`, `sast-scan.yml`, `security-scan.yml`, `security-zap.yml`
Propietario del servicio / CAB	Autoriza ventana, impacto aceptable y decisión final	Docente del curso / Cesar Fabian Chavez Linares, según corresponda a la entrega académica

6. Prerrequisitos y condiciones de entrada

- [ ] Versión candidata identificada (commit/tag) y CI en verde (`testing.yml`, `backend-tests.yml`, `web-tests.yml`, `ai-services-tests.yml`, `mobile-ci.yml`).
- [ ] Código congelado en la rama de despliegue (`fabian` o `main`); build de imágenes exitoso (`docker-build.yml`).
- [ ] `.env` presente en la VM con variables reales (no placeholders) — `scripts/deploy-vm.sh` ya valida automáticamente `MONGO_PASSWORD`, `REDIS_PASSWORD`, `JWT_SECRET`, `JWT_REFRESH_SECRET` antes de desplegar; revisar además `FIELD_ENCRYPTION_KEY`, `CORS_ORIGINS`, `OPENAI_API_KEY` manualmente.
- [ ] Respaldo de `mongodb_data` y `redis_data` (volúmenes Docker) verificado y restaurable.
- [ ] Servicio `tunnel-manager.sh` corriendo en la VM (gestiona `cloudflared` a nivel de host, apuntando a `http://localhost:80`) — confirmar que esté activo antes y después del despliegue, ya que un reinicio de `nginx`/VM puede rotar la URL del túnel.
- [ ] Plan de comunicación enviado (en este contexto: aviso al docente/stakeholders si la ventana afecta una demo o entrega).

7. Procedimiento operativo de implementación

Fase	Actividad / descripción	Evidencia esperada en RespiCare
1. Preparación	Validar checklist, alcance, versión aprobada	Commit/tag confirmado + CI verde en GitHub Actions
2. Salvaguarda	Respaldo de BD, configuración y versión previa; confirmar restauración	`./scripts/backup-vm.sh` — genera `backups/<timestamp>/{mongodb.archive.gz, redis.rdb, .env, .env.vm, git-commit.txt}`

3. Prechequeo	Capacidad, conectividad, permisos, certificados, estado de servicios	Espacio en disco de la VM, estado `docker ps` de contenedores previos
4. Ejecución	Desplegar artefactos aprobados; aplicar variables y migraciones	`./scripts/deploy-vm.sh --build` (backend/ai-services/datos en la VM); push a `fabian/main` para el autodeploy del web en Vercel
5. Verificación	Salud, endpoints, integraciones, pruebas de humo	`./scripts/smoke-test-vm.sh` — healthchecks de los 5 contenedores, `/nginx-health`, `/health`, `/ai/api/v1/health`, `/ai/openapi.json`, y flujo funcional mínimo de `auth/login`; sale con código ≠0 si algo falla (go/no-go automatizable)
6. Habilitación	Restablecer tráfico/accesos y comunicar disponibilidad	URL pública vía `cloudflared` (host) accesible, y build de Vercel del web en estado "Ready"
7. Estabilización	Monitorear errores, latencia, consumo, incidentes	Logs de contenedores (`json-file`, rotación 10m/5 archivos en backend)
8. Cierre	Documentar resultado, desviaciones, evidencias, conformidad	Entrada nueva en este documento o en el sprint correspondiente

8. Cronograma referencial

Hito	Momento referencial	Responsable
Revisión y aprobación	T−1 día	Cesar Fabian Chavez Linares
Respaldo y prechequeos	T−30 a T−10 min	Cesar Fabian Chavez Linares
Implementación	T0 a T+15 min	Cesar Fabian Chavez Linares (`./scripts/deploy-vm.sh --build`)
Pruebas y decisión go/no-go	T+15 a T+30 min	Cesar Fabian Chavez Linares, apoyado en `CATALOGO_PRUEBAS.md`
Monitoreo reforzado	Primeras 24 h	Cesar Fabian Chavez Linares
Cierre formal	T+1 día	Cesar Fabian Chavez Linares

(Tiempos reducidos respecto a la plantilla base porque es un sistema académico de bajo volumen, no una infraestructura empresarial 24/7.)

9. Pruebas y criterios de aceptación

- **Disponibilidad**: los 3 healthchecks de Compose (`backend`, `ai-services`, `nginx`) en estado `healthy`.
- **Funcionalidad**: login/autenticación JWT, registro de síntomas, análisis IA, dashboard epidemiológico completan pruebas de humo manuales o automatizadas.
- **Datos**: si hubo migración de esquema en MongoDB, conciliar conteos de colecciones antes/después.
- **Seguridad**: `CORS_ORIGINS` restringido al dominio real, JWT/encryption secrets no expuestos en logs, `security-scan.yml`/`sast-scan.yml` sin hallazgos críticos nuevos.
- **Rendimiento**: umbrales SLA ya definidos y verificados en CI contra MongoDB real (`backend/tests/performance/api-performance.perf.test.ts`), por tipo de endpoint:

Tipo de endpoint	mean	p95	p99
Auth (login/register)	< 300ms	< 500ms	< 800ms
Lectura (list/get)	< 200ms	< 400ms	< 600ms
Escritura (create/update)	< 350ms	< 600ms	< 900ms
Analytics/Dashboard	< 400ms	< 700ms	< 1000ms
Búsqueda/filtro	< 250ms	< 450ms	< 700ms

Además, `concurrency.perf.test.ts` exige un throughput mínimo de **≥ 5 req/s** en lecturas de historial médico bajo carga paralela, y una degradación bajo carga sostenida **< 3x** respecto al tiempo base. Criterio de aceptación para el despliegue: estos tests de performance deben pasar en CI (`npm run test:performance` en `backend/`) antes de autorizar el paso a producción; `scripts/smoke-test-vm.sh` no mide latencia, solo disponibilidad — si se requiere validar el SLA ya en la VM, correrlo manualmente con `ab`/`autocannon` contra `/health` y `/api/v1/*` tras el despliegue.

- **Operación**: logs rotando correctamente, acceso a `mongo-express`/`redis-commander` solo si es necesario (son herramientas de administración, no deberían quedar expuestas públicamente sin protección).

Criterio de aceptación final: Cesar Fabian Chavez Linares (como responsable único de QA y despliegue) confirma resultados antes de dar por cerrada la liberación. Registrar evidencias en este archivo o en la bitácora del sprint vigente.

10. Riesgos, contingencia y reversión

Riesgo	Mitigación / respuesta específica a RespiCare
Incompatibilidad de configuración	Validar `.env` de producción contra `.env.example`/`docker-compose.vm.yml` antes

	de cada despliegue
Falla de migración o corrupción de datos clínicos	Respaldo de `mongodb_data` antes de cualquier cambio de esquema; nunca ejecutar migraciones sin respaldo verificado
Indisponibilidad o degradación	Desplegar fuera de ventanas de demo/entrega; revertir con `docker compose down` + reinicio de la versión anterior
Problemas de permisos o seguridad	`FIELD_ENCRYPTION_KEY`/`JWT_SECRET` solo en `.env` del servidor, nunca commiteados; revisar `CORS_ORIGINS`
Fallas de integraciones	Verificar conectividad backend ↔ ai-services ↔ MongoDB/Redis antes de habilitar tráfico externo vía Cloudflare Tunnel

11. Procedimiento de rollback

Activadores: healthcheck de algún servicio no pasa a `healthy` tras varios reintentos, error crítico en endpoints principales, pérdida de integridad de datos, o degradación evidente tras habilitar el túnel.

1. El responsable declara "no-go"/rollback y registra la hora.
2. Se detiene el tráfico: `pkill -f "cloudflared tunnel"` o se detiene el servicio de `scripts/tunnel-manager.sh` en la VM para dejar de exponer la versión fallida (el túnel corre a nivel de host, no como contenedor).
3. Se restaura el código al commit estable anterior (`git checkout <commit-estable>` en la VM) y se reconstruye: `./scripts/deploy-vm.sh --build` (las imágenes se buildean en la propia VM, no hay un tag previo en un registro externo que recuperar).
4. Si el push automático de `tunnel-manager.sh` alcanzó a disparar un autodeploy en Vercel con configuración de la versión fallida, revertir también ese deploy desde el dashboard de Vercel o con un nuevo commit que restaure `web/src/utils/apiBase.js`.
5. Si hubo cambios de esquema en MongoDB, se evalúa el impacto antes de restaurar el volumen — **no restaurar sin verificar pérdida de transacciones posteriores al respaldo**.
6. Se verifican salud e integraciones nuevamente, se reinicia `tunnel-manager.sh` para re-exponer el servicio, y se documenta el incidente en este archivo o en `PENDIENTES_CORRECCION_TESTS.md`.

12. Plan de comunicación y soporte

Momento	Destinatarios	Mensaje / canal
Antes	Docente / stakeholders del curso (si aplica a una entrega)	Aviso de ventana y posible indisponibilidad temporal
Inicio	— (equipo de un solo desarrollador)	Registro en este documento

Fin o reversión	Docente / stakeholders	Resultado y hora efectiva, si afecta una entrega programada
Estabilización	—	Revisión de logs en las siguientes 24 h

13. Evidencias y entregables

- Commit exacto desplegado en la VM (`git log -1` tras el despliegue) y, si hubo cambios de web, el deployment ID correspondiente en Vercel.
- Copia del `.env` de producción usado (sin exponerlo en el repo — guardarlo fuera de control de versiones).
- Resultados de pruebas de humo (manual o scriptado) ejecutadas tras el despliegue.
- Capturas o logs de los healthchecks en estado `healthy`.
- Registro de incidencias y decisión go/no-go en este documento.

14. Matriz breve de aprobación

Instancia	Nombre y cargo	Fecha / conformidad
Elaborado por	Cesar Fabian Chavez Linares	23/10/2026
Revisado por (técnico/seguridad)	Cesar Fabian Chavez Linares (autorevisión + CI de seguridad automatizado)	Pendiente por despliegue
Autorizado por (propietario/CAB)	A definir según corresponda (docente del curso o autoaprobación del proyecto)	Pendiente por despliegue

Anexo A. Lista de verificación go/no-go

- [ ] Cambio aprobado y ventana confirmada
- [ ] Respaldo de `mongodb_data`/`redis_data` comprobado y disponible para rollback
- [ ] Código en la VM actualizado al commit aprobado y build de `backend`/`ai-services` exitoso
- [ ] `.env` de producción revisado sin secretos expuestos (validado automáticamente por `scripts/deploy-vm.sh` para las 4 variables críticas)
- [ ] Pruebas críticas ejecutadas y sin fallos bloqueantes (ver `CATALOGO_PRUEBAS.md` — verificar que los fails pendientes en web/ai-services no afecten rutas críticas del despliegue)
- [ ] Healthchecks de `mongodb`, `redis`, `backend`, `ai-services` y `nginx` en `healthy`
- [ ] `tunnel-manager.sh`/`cloudflared` activo en la VM y URL pública accesible
- [ ] Build de Vercel del web (si hubo cambios) en estado "Ready"
- [ ] Autorización expresa para habilitar producción

Anexo B. Procedimiento paso a paso (guía práctica)

Esta guía complementa la sección 7 (procedimiento operativo formal) con los comandos reales, en el orden en que se ejecutan, para desplegar el sistema de principio a fin en la VM de producción.

B.1 Descarga del código

```
ssh <usuario>@38.250.158.233
cd /ruta/al/proyecto
git fetch origin
git checkout fabian
git pull origin fabian
```

B.2 Instalación y configuración

```
# Si no existe .env, se parte de la plantilla de la VM
cp .env.vm .env   # scripts/deploy-vm.sh lo hace automáticamente si .env falta

# Completar manualmente los secrets reales (nunca placeholders):
# MONGO_PASSWORD, REDIS_PASSWORD, JWT_SECRET, JWT_REFRESH_SECRET,
# FIELD_ENCRYPTION_KEY, CORS_ORIGINS, OPENAI_API_KEY
nano .env

# Construir las imágenes de backend y ai-services en la propia VM
docker compose -f docker-compose.vm.yml build --no-cache backend ai-services
```

> *`scripts/deploy-vm.sh` valida automáticamente que `MONGO_PASSWORD`, `REDIS_PASSWORD`, `JWT_SECRET` y `JWT_REFRESH_SECRET` no queden como placeholders antes de continuar; el resto de variables se revisa manualmente.*

B.3 Subida/carga de datos

Con `mongodb` y `redis` arriba (`docker compose -f docker-compose.vm.yml up -d mongodb redis` si aún no están levantados):

```
cd backend
npm install
npm run seed:demo             # usuarios y datos de demostración
npm run seed:health-centers   # catálogo de centros de salud (RF-012)
npm run seed:alerts           # alertas de ejemplo (RF-009), opcional
cd ..
```

> *`seed:demo` es solo para datos de demostración. En un despliegue con datos clínicos reales, omitir este paso y restaurar desde un respaldo verificado (`scripts/backup-vm.sh` / `mongorestore`) en su lugar.*

B.4 Ejecución del sistema

```
./scripts/deploy-vm.sh --build
```

El script verifica Docker, valida `.env`, construye las imágenes, levanta `mongodb`, `redis`, `backend`, `ai-services`, `nginx`, `mongo-express` y `redis-commander`, y espera a que cada contenedor reporte `healthy`. Al finalizar imprime:

```
Backend API:  http://38.250.158.233/api/
AI Service:   http://38.250.158.233/ai/
Health:       http://38.250.158.233/health
Dominio:      http://respicare.sytes.net
```

El frontend web **no** se despliega con este script: se actualiza solo mediante el push a `fabian/main`, que dispara el autodeploy de Vercel.

Para exponer el sistema públicamente (si `tunnel-manager.sh` no está ya corriendo en la VM):

```
nohup ./scripts/tunnel-manager.sh > /dev/null 2>&1 &
```

> *Es un proceso en primer plano que debe quedar corriendo en segundo plano (o como servicio) de forma continua: si se detiene, el túnel público deja de estar disponible.*

B.5 Pruebas

```
./scripts/smoke-test-vm.sh
```

Verifica los healthchecks de los 5 contenedores, los endpoints públicos vía nginx (`/health`, `/ai/api/v1/health`, `/ai/openapi.json`) y un flujo mínimo de `auth/login`; sale con código ≠0 si algo falla.

Para una validación más completa de la suite automatizada, usar `docs/testing/GUIA_QA_EJECUCION_CLI.md` y `docs/testing/CATALOGO_PRUEBAS.md`.

Gaps detectados al aplicar esta plantilla al proyecto real

7. ~~No existe script de respaldo pre-despliegue automatizado~~ — **resuelto**: `scripts/backup-vm.sh` (mongodump + RDB de Redis + copia de `.env`/`.env.vm`, con retención de 14 días).
8. ~~No existe script de smoke test funcional consolidado~~ — **resuelto**: `scripts/smoke-test-vm.sh` (healthchecks + endpoints vía nginx + flujo mínimo de `auth/login`).
9. ~~`deploy-production.yml` (Kubernetes) desalineado con la infraestructura real~~ — **resuelto**: nunca se ejecutó (0 corridas, 0 tags, sin secret `KUBECONFIG_PRODUCTION`) y se archivó en `docs/archive/workflows/deploy-production.yml` con una nota explicando por qué.
10. ~~No hay umbrales de rendimiento definidos~~ — **resuelto**: la sección 9 ahora documenta el SLA real ya verificado en CI (`backend/tests/performance/api-`

`performance.perf.test.ts` y `concurrency.perf.test.ts`): mean/p95/p99 por tipo de endpoint y throughput mínimo ≥ 5 req/s.

11. **El túnel de Cloudflare es efímero** (`*.trycloudflare.com`, no un túnel nombrado/persistente): la URL pública rota en cada reinicio de `cloudflared`/VM, y `tunnel-manager.sh` reacciona con un **commit + push automático a `fabian`** que reescribe `.env`, `nginx.vm.conf`, configs de mobile y `web/src/utils/apiBase.js`. Es una automatización sin revisión humana que debería, como mínimo, quedar documentada como tal (y evaluarse si conviene migrar a un túnel nombrado de Cloudflare para tener una URL estable).
12. ~~Discrepancia de IP~~ — **resuelto**: `nginx/nginx.vm.conf` tenía `38.250.158.226` desactualizado; se corrigió a `38.250.158.233`, consistente con `.env.vm`, `nginx/nginx.server.conf` y lo que imprime `scripts/deploy-vm.sh`.